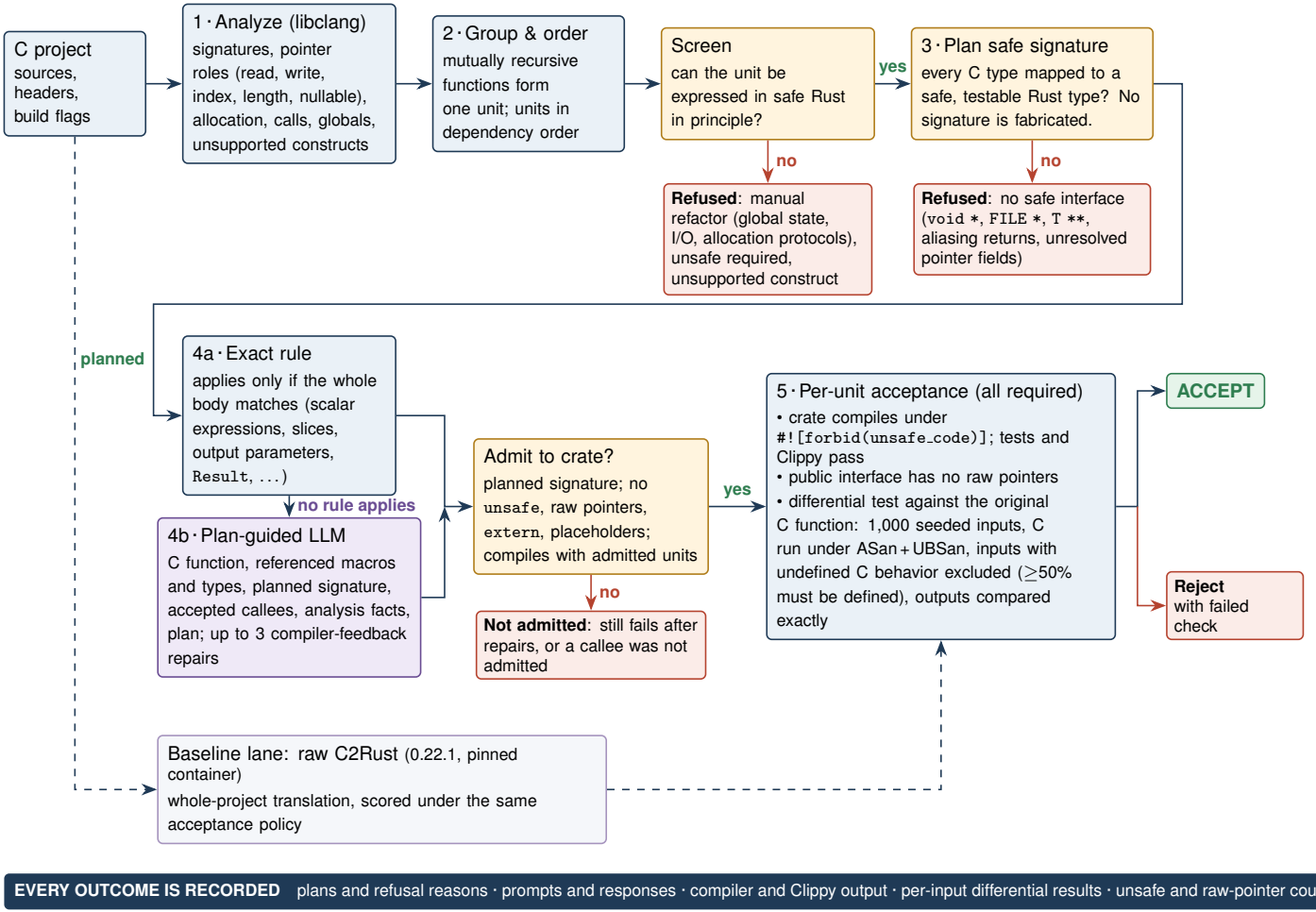
C project
sources, headers, build flags
1 · Analyze (libclang)
signatures, pointer roles (read, write, index, length, nullable), allocation, calls, globals, unsupported constructs
2 · Group & order
mutually recursive functions form one unit; units in dependency order
Screen
can the unit be expressed in safe Rust in principle?
yes
no
Refused: manual refactor (global state, I/O, allocation protocols), unsafe required, unsupported construct
3 · Plan safe signature
every C type mapped to a safe, testable Rust type? No signature is fabricated.
no
Refused: no safe interface (void *, FILE *, T **, aliasing returns, unresolved pointer fields)
planned
4a · Exact rule
applies only if the whole body matches (scalar expressions, slices, output parameters, Result, ...)
no rule applies
4b · Plan-guided LLM
C function, referenced macros and types, planned signature, accepted callees, analysis facts, plan; up to 3 compiler-feedback repairs
Admit to crate?
planned signature; no unsafe, raw pointers, extern, placeholders; compiles with admitted units
yes
no
Not admitted: still fails after repairs, or a callee was not admitted
5 · Per-unit acceptance (all required)
• crate compiles under #![forbid(unsafe_code)]; tests and Clippy pass
• public interface has no raw pointers
• differential test against the original C function: 1,000 seeded inputs, C run under ASan + UBSan, inputs with undefined C behavior excluded (≥50% must be defined), outputs compared exactly
ACCEPT
Reject with failed check
Baseline lane: raw C2Rust (0.22.1, pinned container)
whole-project translation, scored under the same acceptance policy
EVERY OUTCOME IS RECORDED
plans and refusal reasons · prompts and responses · compiler and Clippy output · per-input differential results · unsafe and raw-pointer counts